

Quantification of Cost-Resistance Tradeoff in Password Hashing Algorithms

Christoffer Larsen

Integrated Masters 3.year

UiT The Arctic University of Norway

Tromsø, Norway

chlar7639@uit.no

chlar7639@github

Deividas Kalnikas

Department of Computer Science

UiT The Arctic University of Norway

Tromsø, Norway

dekal0739@uit.no

ImmortalStogh@github

I. INTRODUCTION

Password security still largely comes down to one question, how the system stores and verifies authentication keys. Storing passwords in plaintext or simple hashing functions leaves the system exposed because of low entropy and randomness. Today ASIC hardware and GPU can compute billions of such hashes in seconds, making brute-force and dictionary attacks easy for password cracking. To address this, password specific hashing algorithms were created like SHA-256 and BLAKE3 to be slow and resource intensive. Making brute-force attacks computationally expensive.

This project investigates and compares the most widely used password hashing algorithms including SHA-256, BLAKE3, Argon2id, scrypt, bcrypt, and PBKDF2 with a focus on their cost parameters and how those parameters influence both system performance and resistance to attack.

A. Motivation and goal

The primary challenge in password hashing is compensating two competing demands. On one hand, the hashing process must remain fast enough that users experience acceptable login latency. On the other hand, it must be slow and resource intensive enough that attackers attempting to crack stolen password hashes typically using GPU accelerated brute force or dictionary attacks face a significant computational barrier. Every Resource parameter (such as memory usage, latency, iteration count and parallelism) sit somewhere on this trade off curve, and choosing the wrong configurations can either frustrate users or leave the system vulnerable.

This matters because password breaches remain one of the most common and damaging security incidents, and the choice of hashing algorithm and configurations is often the last line of defense once a database has been compromised. Despite this many still use outdated hashing algorithms. This project aims to provide practical, evidence based on selecting and tuning a hashing algorithms that offers the best resistance against attackers within an acceptable latency budget.

B. Research questions

Several research questions are put into place:

- Question 1: How will each algorithm's parameters affect the authentication latency?
- Question 2: How will the parameter cost affect the attacker's cracking throughput (guess/sec) on GPU hardware?
- Question 3: Given an "acceptable" latency budget, which combination of parameter and algorithm will provide the best resistance against a given time budget?

C. Hypotheses

The hypotheses set in place are:

- Hypothesis 1: Memory-hard algorithms will have a much steeper GPU-cracking slowdown per unit of legit-login latency cost than CPU-hard algorithms, because GPUs excel at parallel computing, but at memory limited.
- Hypothesis 2: CPU-hard algorithms will be cracked at a meaningfully higher throughput than the GPU based, even at equivalent time limits, thus exposing memory-hardness as the dominant factor rather than time cost.

II. RELATED WORK

Research on hashing methods has established that compared PBKDF2, Bcrypt, Scrypt in terms of implementation and performance, evaluation factors like execution time and resistance to brute force/hardware attacks [2]. The study was limited to a specific set of parameters/hardware and did not address real world deployment.

Biryukov, Dinu & Khovratovich [1] introduced Argon2, the winner of the password hashing competition, as a new generation of memory hard functions designed to resist GPU/ASIC cracking attacks. There are two variants (Argon2d and Argon2i) balancing resistance to GPU attacks against resistance to side-channel attacks, along with tunable cost parameters. Strong theoretical and competition backed grounding, but as a foundational paper, it focused on a more broader view rather than empirical comparison against other hashing algorithms in real world deployment context.

Pandya [3] evaluated the performance of three widely used hashing algorithms, BLAKE3, SHA-256 and SHA-512 in context of blockchain application, benchmarking hashrate and latency across multiple platforms. It was found that BLAKE3 generally outperforms both SHA-256 and SHA-512 in throughput and latency, though the performance advantage varies depending on the platform and data size. The study offers useful cross platform benchmarking but focuses generally on blockchain hashing functions rather than password hashing algorithms specifically, and doesn't address resistance to brute force attacks.

Sprengers sets up the and provides the numerical data for the "GPUs broke the old assumptions" narrative, justifying why this is a real problem [5], while Prokarovsky et al. established a good methodology precedent using the same tools and the comparison structures as planned for this project [4].

III. SYSTEM

The system will consist of a *sweep controller* that will initiate, define parameters and iterate runs. The hashing algorithm hashes the password with it's set parameters and records the latency, after that it is given to the attacker pipeline and attempted to be cracked within a given time. Regardless of the results, the metrics are recorded and tagged.

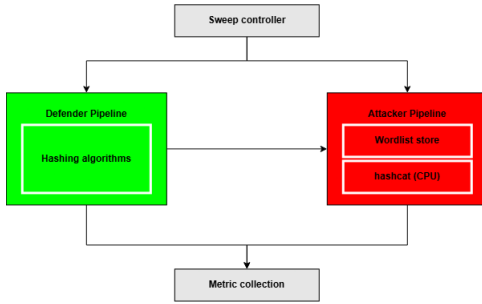


Fig. 1. Component diagram showcasing the system for the metric

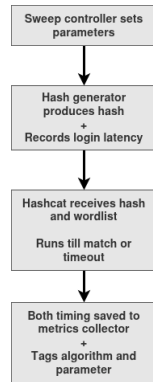


Fig. 2. Sequence diagram showcasing the sequence in which the metric will be collected in.

IV. EARLY ENVIRONMENT AND SCOPE

The experiments will run in an isolated, offline test environment to avoid any risk to live systems or real credential data. Password samples will be either generated and drawn from publicly available data, salting is integrated into the hashing algorithms.

Benchmark machines representing a defender and attacker roles, will be used to measure our research questions. Ideally a machine with a multi core CPU, to simulate realistic behavior from server when it comes to hashing, memory distribution and parallelism. The project does not require any external network or production access, as testing is selfcontained.

V. EARLY TIMELINE

The setup and literature review will take 1-3 weeks. Baseline implementation will take 1-2 weeks. Benchmarking could will take approximately 4 weeks for latency and attacker throughput benchmarking. Then about 4 more weeks for cross analysis, report writing and presentation. These numbers can be changed to fit the agenda.

VI. DISCUSSION

Consumer grade GPUs used for testing may not reflect the capabilities of well funded attackers using clusters of high end GPUs showing the lower absolute guesses/sec, but the relative comparisons between algorithms still holds, which is what the hypotheses depend on.

Six algorithms * multiple parameters * two measurement dimensions(latency and throughput) is a substantial testing amount. There's a risk of insufficient time to complete full sweep for all algorithms. a fallback plan would be to prioritize some of the algorithms and change the amount of testing to fit the time limit.

Benchmark results can be sensitive to background system load, heat or driver versions. Running controlled tests, contained environment and repeating each measurement multiple times will help mitigate noisy or non-reproducible results.

REFERENCES

- [1] Alex Biryukov, Daniel Dinu, and Dmitry Khovratovich. "Argon2: New Generation of Memory-Hard Functions for Password Hashing and Other Applications". In: *2016 IEEE European Symposium on Security and Privacy (EuroSP)*. 2016, pp. 292–302. DOI: 10.1109/EuroSP.2016.31.
- [2] Levent Ertaul, Manpreet Kaur, and Venkata A. K. R. Gudise. "Implementation and Performance Analysis of PBKDF2, Bcrypt, Scrypt Algorithms". English. In: *Proceedings of the International Conference on Wireless Networks (ICWN)*. 2016, pp. 66–72. URL: <https://www.proquest.com/conference-papers-proceedings/implementation-performance-analysis-pbkdf2-bcrypt/docview/1817061029/se-2>.

- [3] Marut Pandya. *Performance Evaluation of Hashing Algorithms on Commodity Hardware*. 2024. arXiv: 2407.08284 [cs.CR]. URL: <https://arxiv.org/abs/2407.08284>.
- [4] Stefan Pocarovsky et al. “Computational Efficiency of Brute-Force Attacks on Hashing Algorithms”. In: *Software Engineering: Emerging Trends and Practices in System Development*. Ed. by Radek Silhavy and Petr Silhavy. Cham: Springer Nature Switzerland, 2026, pp. 402–413. ISBN: 978-3-032-00236-5.
- [5] Martijn Sprengers. “GPU-based password cracking”. In: *On the Security of Password Hashing Schemes regarding Advances in Graphics Processing Units*, Radboud University Nijmegen (2011).